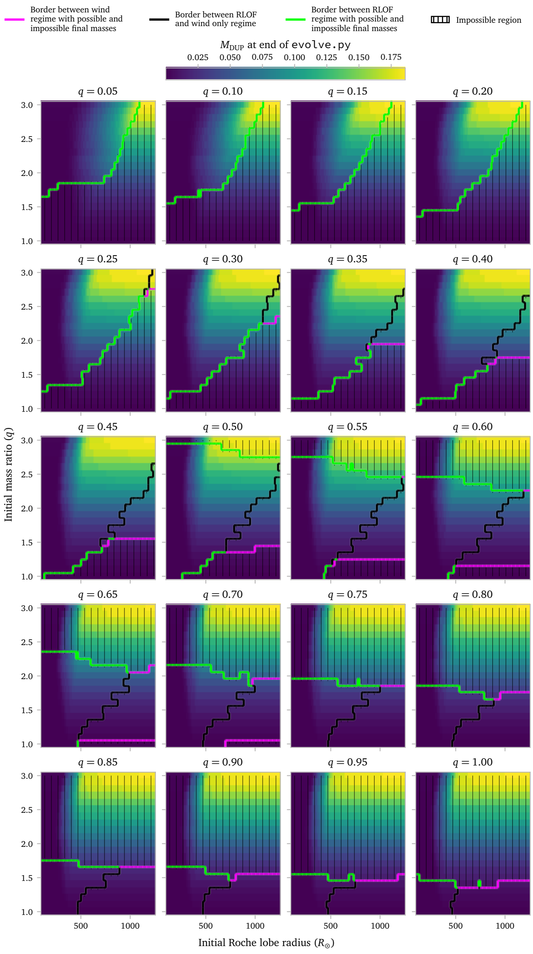
Notes Week 24

a. Grid

a.1 Grid setup

I want to generate a 3 dimensional grid where models have a significant amount of dredged up mass in their envelope when they undergo RLOF.

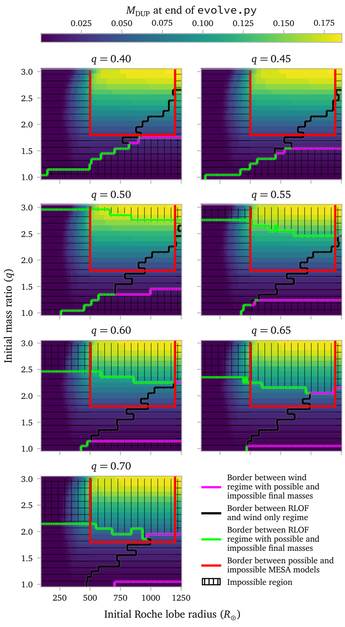
I think it is interesting to vary the masses, initial mass ratios and the initial Roche lobe radius. According to the picture below from last week, we require a minimum Roche lobe radius of *at least* 500 $R_\odot$ for RLOF with significant amounts of dredge up.



The picture also shows us that we need a donor mass that is initially heavier than about 1.8 $M_\odot$.

We can also exclude the regions where $q < 0.4$ since these are *really* difficult to simulate with *MESA*.

Together with the less interesting regions that create masses outside our sample of observations, we obtain the following regions.



This figure shows the regions that I chose as a first *MESA*-grid try. They are the systems that lie inside the red rectangle. even if parts of these regions contain vertical stripes.

Clearly this is only a small region of the total possible parameter space, *but* I need to make sure the simulations can run within a reasonable time.

For the larger q values, many of the produced barium stars will likely be too heavy. This means we cannot directly compare them with the observed sample, *but* these stars likely do exist.

Even though grid regions are marked by a red rectangle, I do exclude *some* of the models in this rectangle. This is if they do not undergo RLOF, in which case we can simply compute their wind mass-transfer without the need of MESA binary evolution.

I am limited by the specific masses that I have simulated with *MESA*, meaning I cannot *easily* simulate *MESA* models with a TPAGB star of mass 1.05 $M_\odot$.

The pictures above show that the masses between $M_{\rm TPAGB} = 3.0\ M_\odot$ and $M_{\rm TPAGB} = 1.8\ M_\odot$ are most relevant for significant amounts of dredged up mass. I want to test the masses $[1.8, 2.2, 2.6 \text{ and } 3.0]$[1].

This is *not* the case of the values of $q, R, \varepsilon, \delta$ and β.

In this case, I think the Roche lobe radii can quite easily be split up into 8 values: $[500, 600, 700, 800, 900, 1000, 1100 \text{ and } 1200]$.

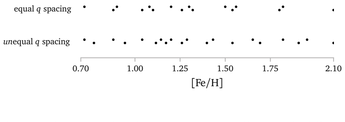
In order to keep the grid to a relatively manageable size, I will only simulate a single value of ε, which is 0.25. This is exactly between the minimum and maximum value of ε that I used in the pictures above.[2]

I will use $\delta = 0.2$. This, again is an assumption which only changes the amount of angular momentum lost. It does not change the accretion[2]. Since $\varepsilon = 0.25$ and $\delta = 0.2$, we must use $\beta = 0.55$.

Lastly, we need to choose q values. An intuitive choice is to take q between 0.4 and 0.7 in equal steps, which would lead to $0.4, 0.5, 0.6$ and 0.7.

I use this.

I also think it could be interesting to q values to space out the initial accretor masses.



The equal q spacing has 4 values, $0.4, 0.5, 0.6$ and 0.7. There are large gaps, especially at the large masses.

The unequal q spacing has 5 values, $[0.4, 0.43, 0.64, 0.65, 0.7]$. Here the gaps are much smaller.

The *un*equal q spacing is based on a maximum interval between the different initial accretor masses.

I test both options, which is possible because they use the same limits on q, which saves on the amount of *MESA* simulations.

They can also be combined later, since the intermediate q values are different.

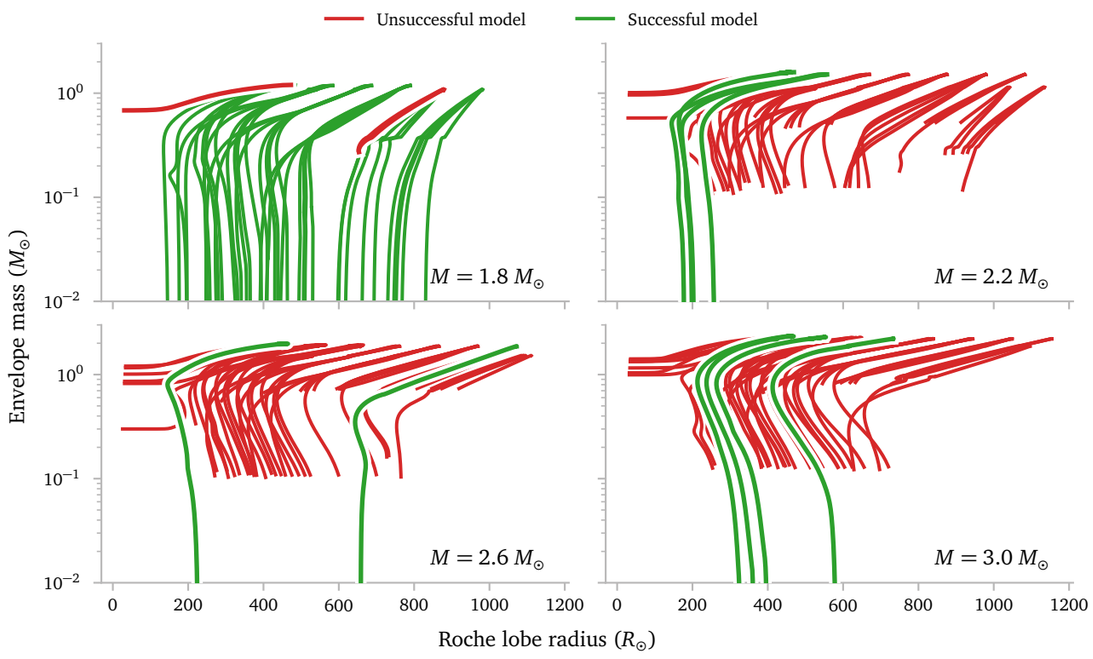
1 Preferably we would use a denser mass spacing, but to keep the size of the grid manageable, I opt to only use 4 here.

2 This is an obvious variable to vary in a future, more advanced grid.

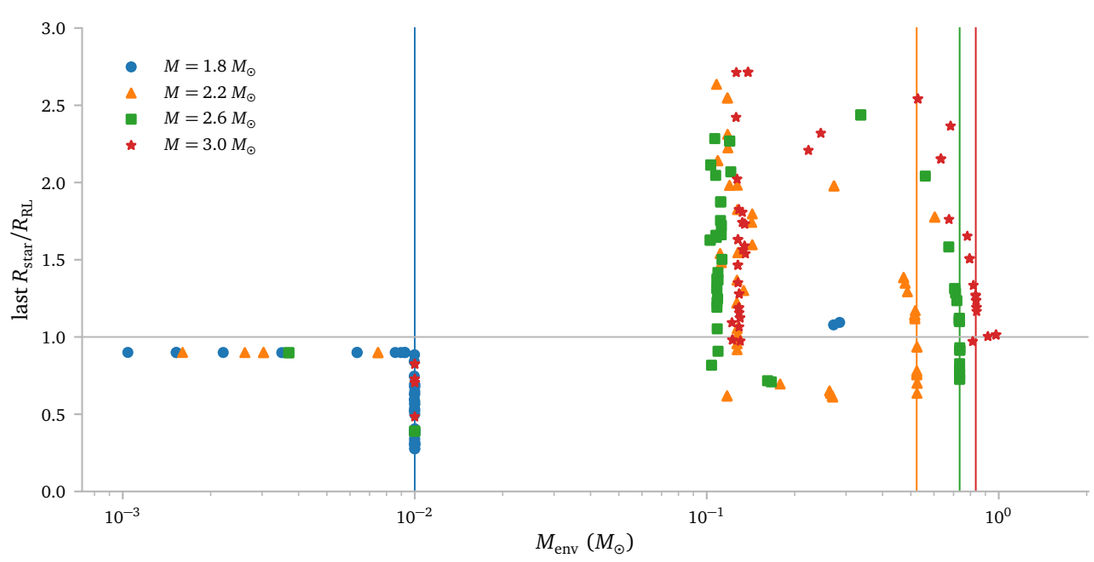
3 Again, this is a variable that *could* be varied in a future grid as well

a.2 Grid results

Let me just start by saying that this grid was *not at all* successful. Of the 194 models, only 47 successfully evolved through RLOF and stripped their envelope. Of these 47, 37 are from the $M = 1.8\ M_\odot$ single star model.



The plot above shows which models successfully finished their evolution and which models failed.



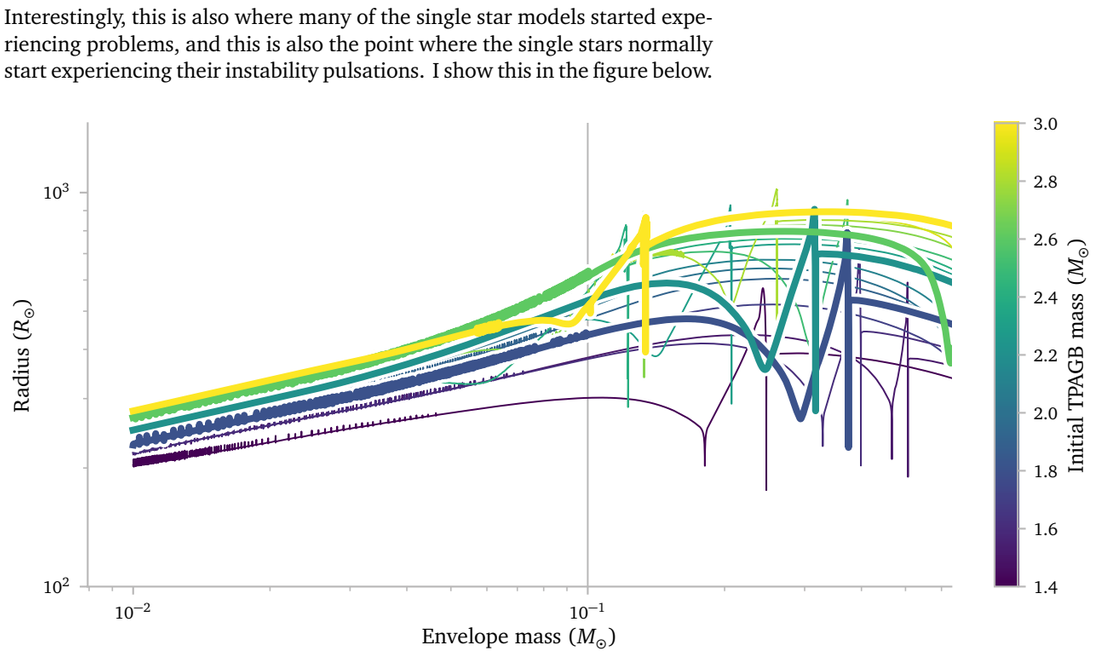
I also show 4 vertical lines here, these correspond to the points at which the single stars stop without manual intervention.

It is also important to note that I excluded the models here that stopped due to the Darwin instability, which lie much higher in this plot.

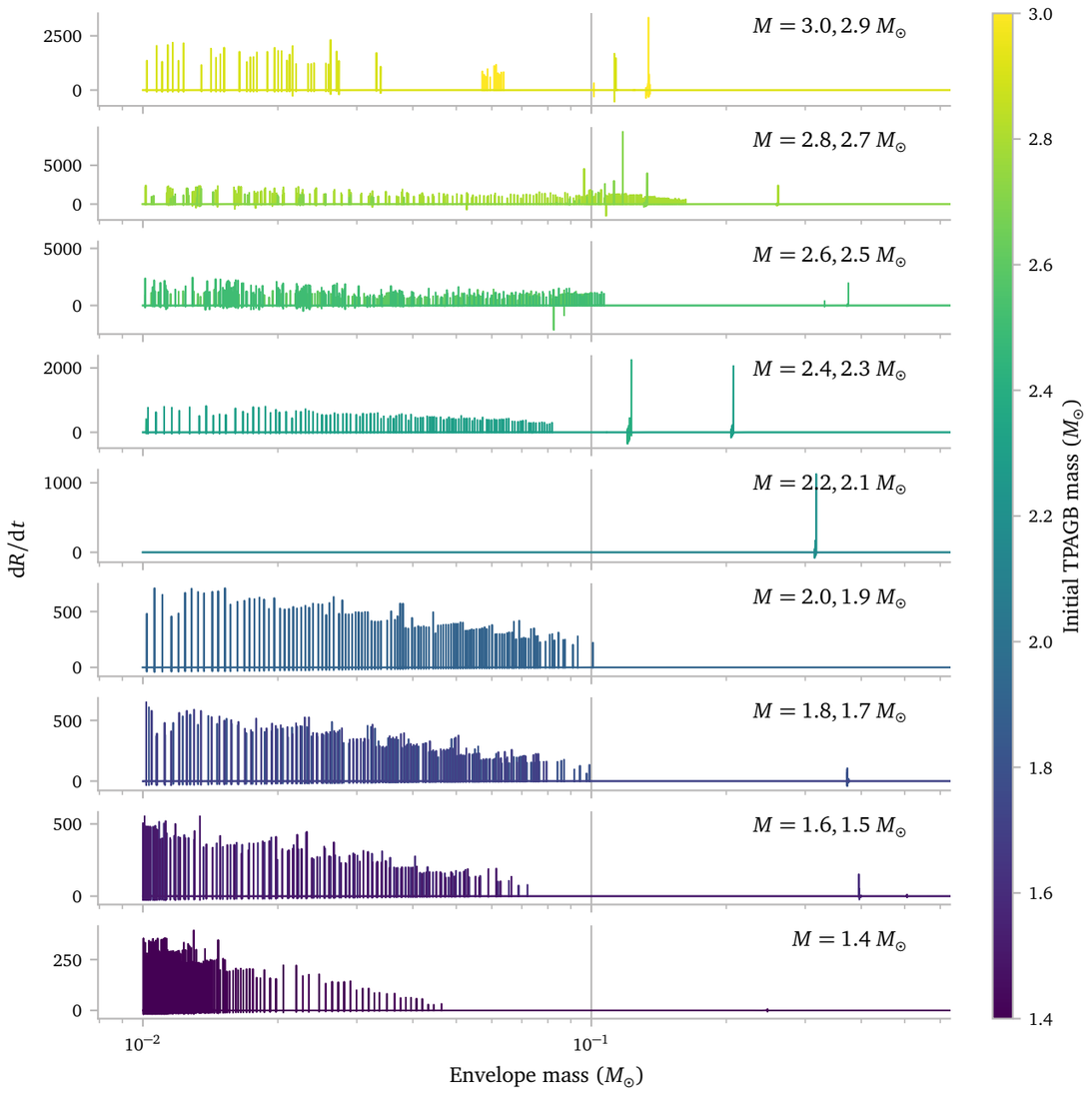
Unfortunately, it seems like the models from the stars that cannot strip their envelope without a companion without crashing, also have a lot of trouble *with* a companion. On hand hand, this makes sense. On the other hand, it does not. The envelope conditions of the stars during RLOF, which a lot of stars are clearly in when they experience instabilities and crash, are quite different from a regular single star envelope.

For some reason, *a lot* of models seem to experience difficulties in passing the 0.1 $M_\odot$ envelope mark.

Interestingly, this is also where many of the single star models started experiencing problems, and this is also the point where the single stars normally start experiencing their instability pulsations. I show this in the figure below.



To show the pulsation a bit clearer, I plot the derivative of the radius with respect to time:



Unfortunately, especially the start of these numerical instabilities can be quite difficult for MESA to evolve through. I found some information online, which I will copy below.

> Dear Mesa users,
>
> I'm trying to evolve stars with mass varying between 1-5 Msun from ZAMS to white dwarf cooling track with version r25.12.1. However, I encounter numerical difficulties during the late TP-AGB phase for a 5Msun model with Z=0.012 (remaining mass 1.25 Msun). The first thermal pulses go smoothly, but the late evolution struggles and the timestep becomes small and goes into a loop of increase/decrease between -3 < log dt < -2.5. The limiting factor for dt is labelled as max increase until a retry and loops like this. I tried to lower `max_timestep_factor` to 1.01 but it does not solve the issue. I observe a lot of retries during that part of evolution until crash due to `min_timestep_limit`.
>
> I am fairly new to mesa and not very experienced with debugging. Here are typical examples of error messages I get:
>
> ```
> avg resid  0.223E-06     max resid dv_dt     1  0.35367E-03 mix type xx000 avg corr
>  ↪  0.474E-03      max corr lnd   451 -0.21100E-01 mix type 00000  avg+max corr+resid
>  ↪  -- give up
>
> hydro_mtx: logT too small       26764         407           3
>  ↪  6.7237179921904777D-01    5.3757388036508846D+00
>
> hydro_mtx: logRho too large       26738         421           5
>  ↪  5.6268674362592407D+01   -7.8503397605837941D+00
> ```
>
> I tried to include `okay_to_reduce_gradT_excess = .true.` which I found used in some inlists online, but with this parameter activated the star shows non-physical behavior (log L > 10$\hat{5}$-6 with few small timesteps) so I keep it to false.
>
> Please find below the enlists I use which are based on `make_co_wd test_suite` which I modified to include element diffusion during MS and thermal pulses to remove the enveloppe instead of `remove_env` routine. Thanks a lot!
>
> Best regards,
>
> *Théo*

> Hi Theo, The late TP-AGB is a very tricky evolutionary stage, especially at higher masses. I'd recommend having a look at Rees & Izzard (2024), as they go through the main instabilities TP-AGB models experience, and offer possible solutions as mesa subroutines. Given the max residuals at the surface and the envelope mass, you might be encountering HRI .
>
> However, their subroutines might not (and at higher masses certainly won't) solve your problem, merely delaying it to a later point in the TP-AGB. If your interest lies *in* the TP-AGB stage, the default approach is to carry calculations on for as long as possible and go through post-processing. If you're interested in the WD stage, the only workaround is to remove the envelope before the instabilities crash the code.
>
> Good luck!
>
> *Ana Antonini*

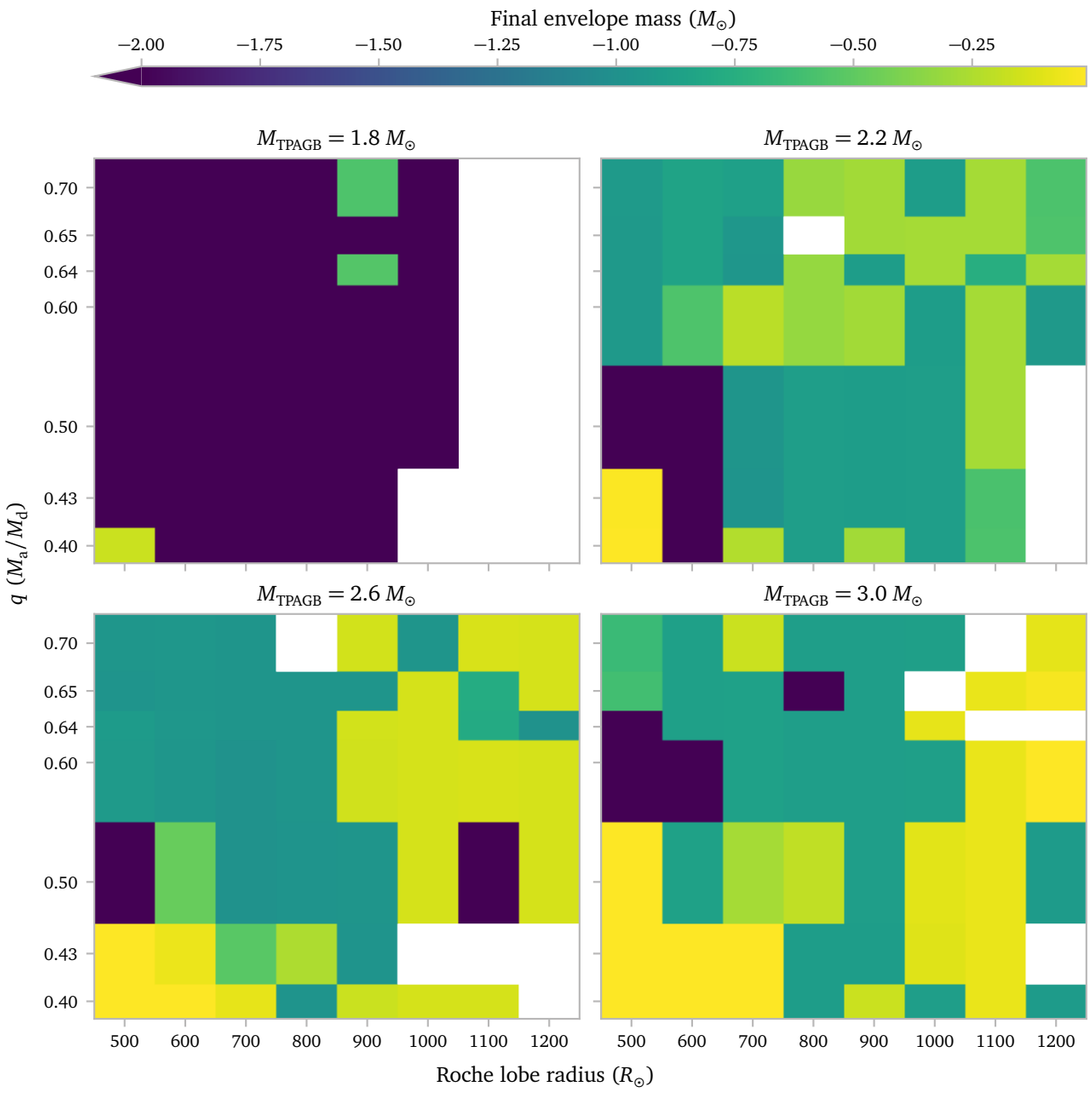
> > Hi Ana,
> >
> > Thanks a lot for your answer! I'll definitely look at that paper. I'm primarily interested in the white dwarf stage, however when using the `make_co_wd` test suite (with the `remove_env` routine) I cannot reproduce the initial-final mass relation of https://ui.adsabs.harvard.edu/abs/2016ApJ...823..102C/abstract and https://academic.oup.com/mnras/article/527/2/3602/7420520. The final mass is significantly underestimated, especially around 2-3Msun (0.5-0.6 Msun vs expected 0.6-0.7Msun). At higher masses it seems to fit better however. I went through the TP-AGB route to use a more physically accurate mass loss scheme to correct that issue. Anyway, thanks again for your reply, I'll try to find some workarounds.
> >
> > Best,
> >
> > *Théo*

> > > Have you checked that you are using the same convective treatments and parameters as the MIST paper? Also, the mass-loss rates you are getting during the AGB with `eta_B` might be too high for the lower masses. You can start the evolution with a smaller factor (or by limiting the mass-loss rate with the "`max_wind`" control). At the higher masses, just make sure you have modest mass loss until the end of the 2DU, and maybe allow 3-5 pulses. After that, the core growth rate gradually decreases, so the effect of extra pulses on the WD mass will be small, and you can try to increase mass-loss to force the models out of the TP-AGB stage.
> >
> > It is always worth remembering that the `test_suites` are precisely that, tests, and are not able to reproduce all the aspects of more sophisticated models.
> >
> > *Ana Antonini*

> > > hi théo,
> > >
> > > mesa can evolve through all the thermal pulses to a wd (for example, as done in https://scixplorer.org/abs/2022ApJ...935...21C/abstract), but the cost is to use very small time steps that can result in run times of order a week or longer. that your timestep is flopping around between -3 < log dt < -2.5 is a large clue that the timesteps are too large. the best way to decrease the timestep (https://docs.mesastar.org/en/25.12.1/reference/controls.html#timestep-controls) is to limit changes in the relevant physical quantities (not numerical quantities). for example, experiment with significantly lowering the default values for changes in the density `delta_lgRho_limit`, temperature `delta_lgT_limit`, luminosity `dL_div1_limit`, helium luminosity `delta_lgL_He_limit`, and the major elements of the composition `dX_limit`. you've found the right combination when the convergence of a single timestep takes ∼ 3 newton iterations and the timestep is varying smoothly and slowly. with patience for longer run times, and a little determination, it can be done!
> > >
> > > *Frank Timmes*

> Hi Théo,
>
> There is a rich literature on the difficulty and ultimatel inability to evolve TP-AGB stars realistically with 1D stellar evolution codes, going back half a century. Alan Sweigart was on of the early contributors to this field.
>
> With a 1D stellar evolution code we need to keep in mind that it is genuinely blind to 3D macro physics. The 1D version of the conservation laws can't see 3D macro physics. TP-AGB stars with their fluffy envelopes, with mass out into the shallow wings o the gravitational potential, and with their global convection morphologies and pulsations are dominated by 3D macro physics.
>
> A 1D code not converging may not be due to a bug or due to a weak numerical scheme or even due to a poor choice of insist parameters. A 1D code not converging may just be saying to you: "Don't ask me to do the impossible"
>
> Best,
>
> *Dr Falk Herwig, Professor*

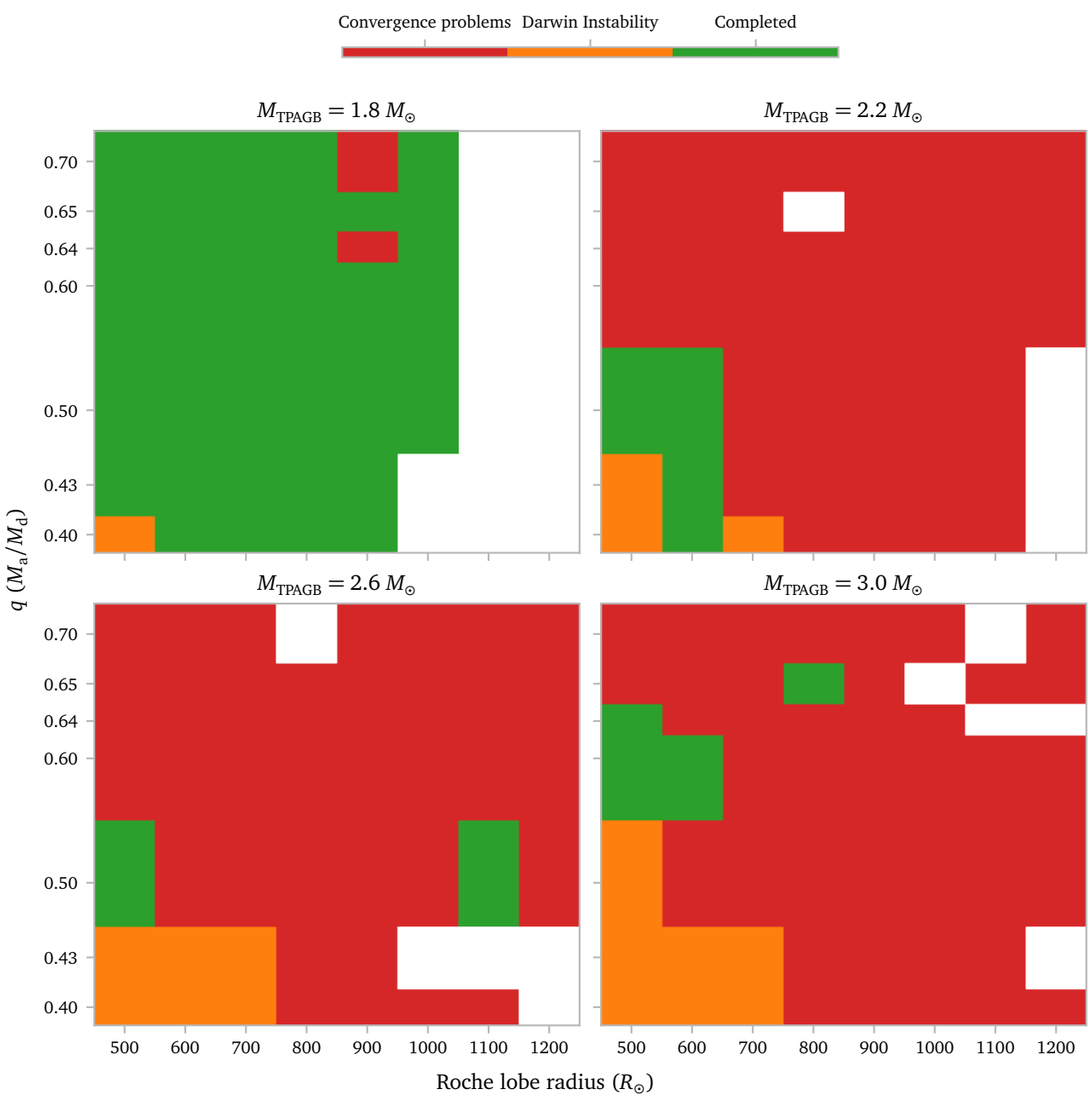
I combine all q values into a single grid. Below I show the envelope mass remaining at the end of the binary evolution.



The white regions are either unsimulated, or missing[4] models. The 1.8 $M_\odot$ grid has a lot of missing models on the right, which is because these models would not undergo RLOF and thus not be very interesting to investigate with MESA.

The see *why* the models did not reach the minimum required envelope mass to terminate, I plot the stopping reason:

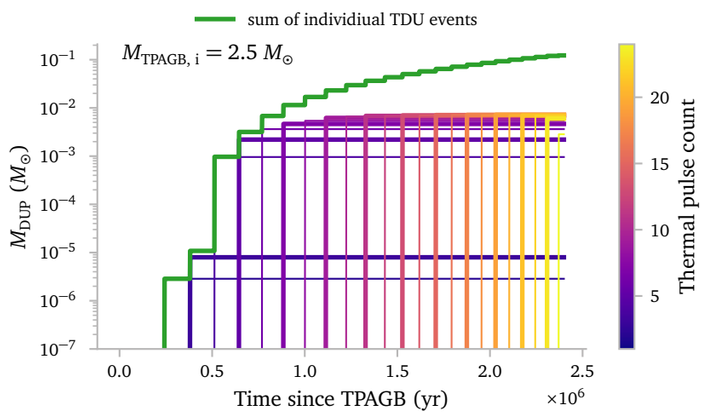
4 For example because they did not finish within 2 days



b. Dredge up

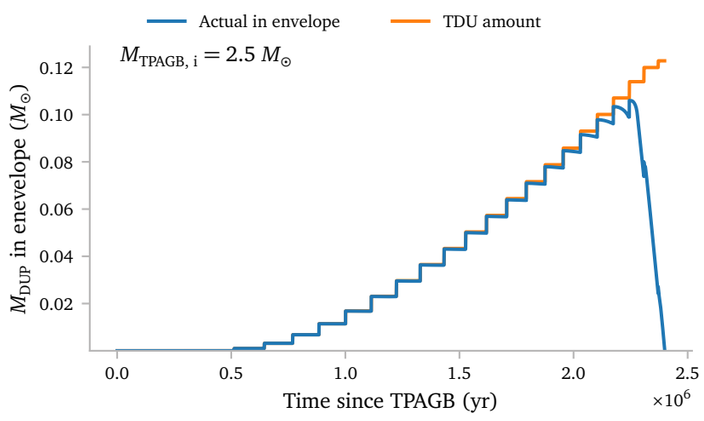
Since I know how much mass is dredged up during every thermal pulse dredge up event, I can compute some interesting things.

To start, we can simply look at the amount of dredged up mass over time.



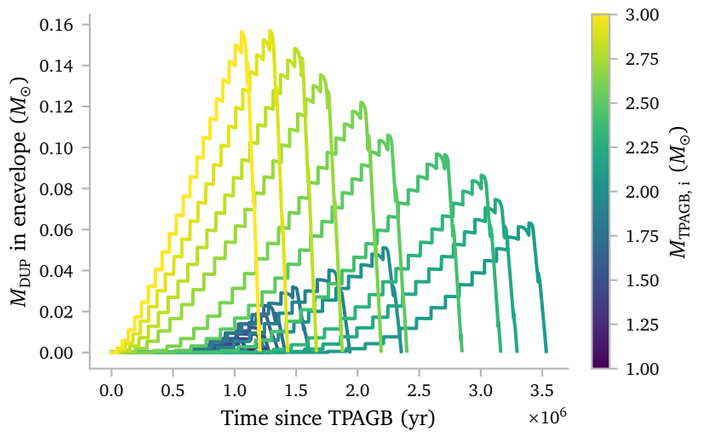
Now obviously, this plot is a little bit boring. The TDU events are so short that they happen basically instantaneously when compared to the total TPAGB lifetime. The amount of dredged up mass obviously also does not change once the dredge up event has concluded.

However, because we know the contributions of the individual TDUs, we *can* look at some other interesting quantities.



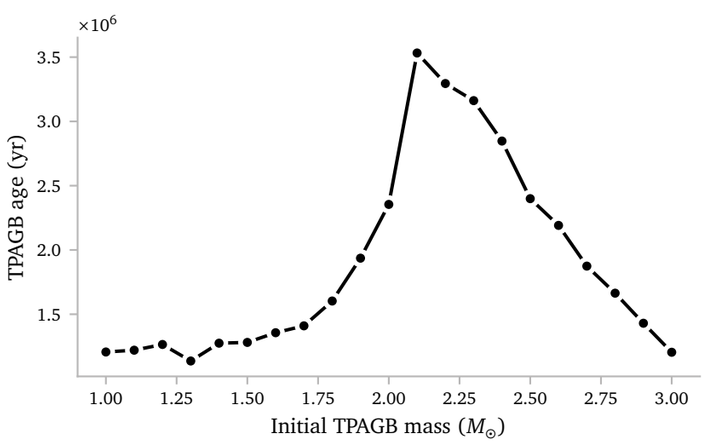
For example, we can see the amount of dredged up mass in the envelope as a function of time, taking into account mass loss from the envelope due to winds.

The actual amount of dredged up mass *in the envelope* is nearly identical to the total amount of dredged up mass up until the wind starts to pick up, after which the amount of dredged up mass decreases.



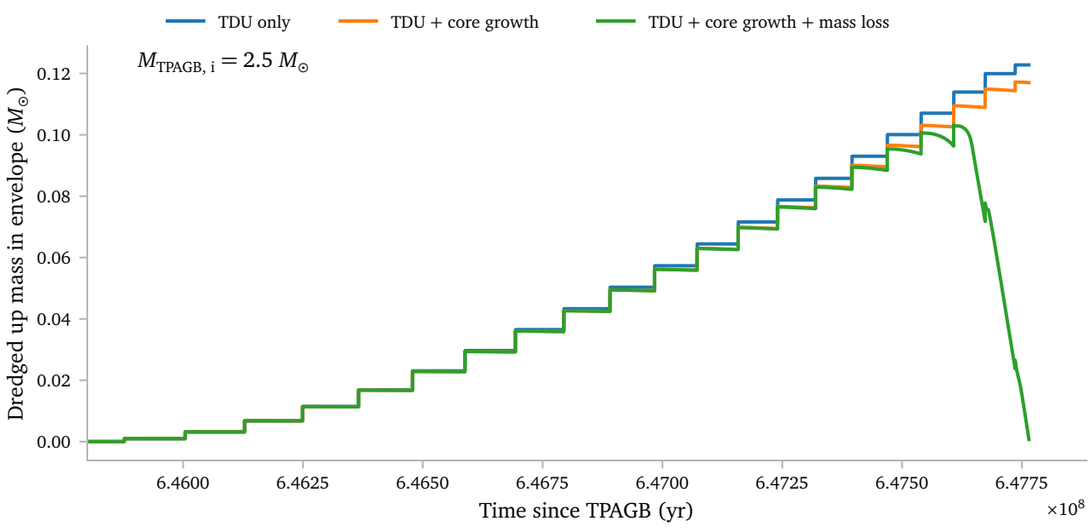
The shape of the M_{DUP} curves are very similar for the different masses, although it is pretty clear that the total amount of dredged up mass is much larger.

Apparently the TPAGB does not keep increasing with mass. Instead, for masses above about $M = 2.2\ M_\odot$, the TPAGB age seems to *decrease*.

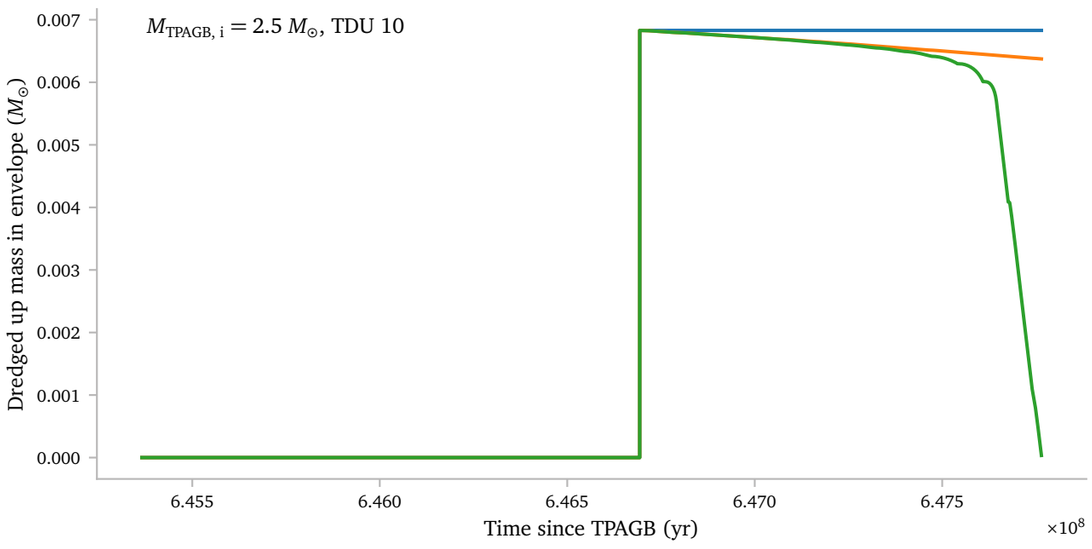


This figure clearly shows this. It seems like it was a coincidence that I only saw the increase in TPAGB age with mass.

> **NOTE:** Above, I showed a picture of the amount of dredged up mass *in the envelope* as a function of time. This quantity decreases over time due to *wind* mass losses, and in the case of a binary evolution, possible RLOF events. *However*, it *also* decreases over time due to the core mass growth, which causes envelope mass to be burned and developed into a dense core. This decrease in envelope does *not* contribute to the possible enrichment of a companion, so it should be separated.



This figure clearly shows the separations. Using my code, I can also inspect how the dredged up mass is removed from the envelope for a single TDU event:



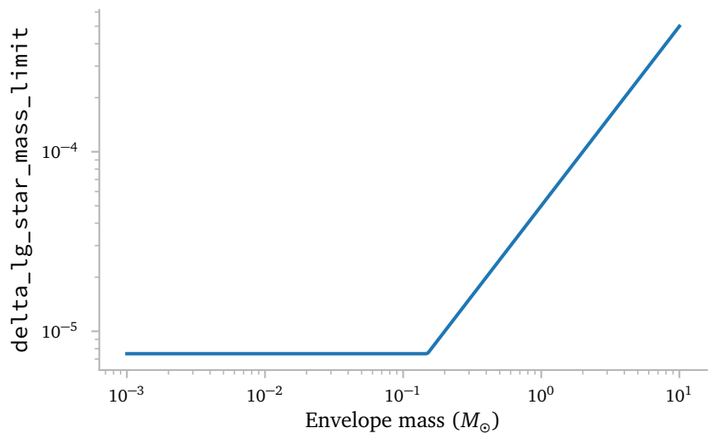
c. Improvements

Here, I describe my attempt at a possible improvement, which is largely based on the reply from Frank Timmes to Theo. He suggests quite drastically lowering changes in physical quantities.

I tried some of his suggested variables, but found that `delta_lg_star_mass_limit` works very well in constraining small changes to the envelope.

By running some tests, I found that lower values of around 10^{-5} work well. It is, however, very wasteful to simulate the entire evolution with such a small Δt. I therefore want to gradually lower the limit.

I devised the following `delta_lg_star_mass_limit` as a function of envelope mass:

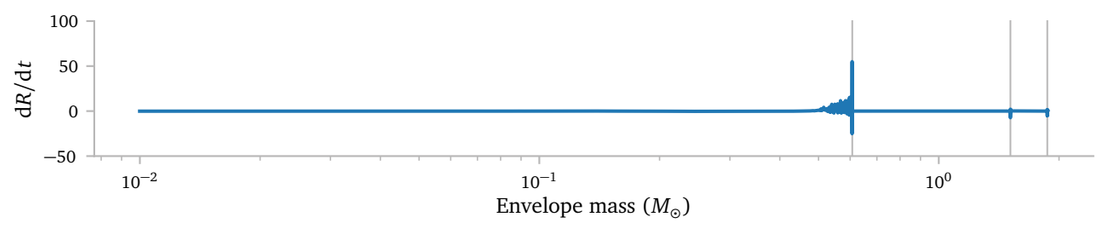


The *MESA* implementation is pretty simple. We can directly change the `delta_lg_star_mass_limit` value based on the current envelope mass.

```
subroutine update_max_dm(s, ierr)
    integer, intent(out) :: ierr
    type (star_info), pointer :: s

    real(dp) :: min_dm = 5.0d-6
    real(dp) :: max_dm = 1.0d-3
    real(dp) :: envelope_mass

    envelope_mass = s% star_mass - s% he_core_mass

    s% delta_lg_star_mass_limit = min( max(envelope_mass*2.0d4, min_dm), max_dm)

end subroutine update_max_dm
```

I simply call this routine at the end of every finished model step.

I have not had time to thoroughly test the full implementation, but a first quick test results in the following plot



The gray vertical lines are the thermal pulses. We see that at those points, we get some large $\mathrm{d}R\,/\,\mathrm{d}t$ values, however, unlike in one of the plots above, we do not see the sharp pulsations when the envelope becomes small.

I hope this fix works for many, if not all of the stars. It could however just be a lucky fix for the model that I tested it on.

The disadvantage of this method is that it obviously causes the star to take significantly smaller timesteps, which also means the binaries will likely take a bit longer to simulate than before.

> **NOTE:** As it turns out, this is *not* sufficient.

I also experiment with increasing the `mesh_delta_coeff` to larger values during the strong wind phases:

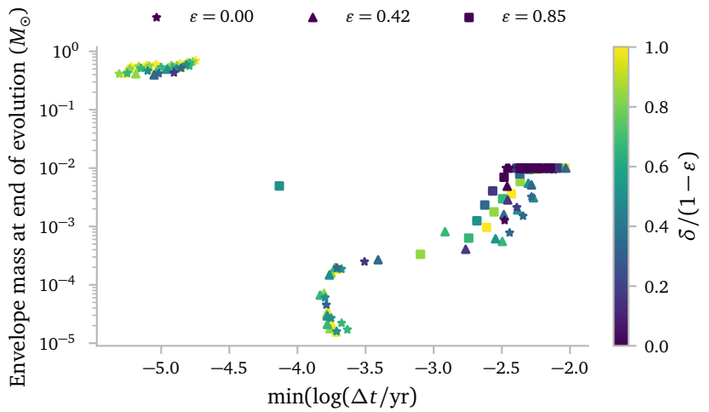
```
subroutine resolve_mesh_during_mass_loss(s, ierr)
    ! Increase mesh_delta_coeff during strong envelope
     ↪  stripping
    ! outside of thermal pulses, to reduce the number of
     ↪  cells.
    !
    ! call in extras_finish_step
    type (star_info), pointer :: s
    integer, intent(out) :: ierr

    real(dp), parameter :: lg_mdot_limit = -5.5d0
    real(dp) :: target_mesh_delta_coeff = 1.0d0

    ierr = 0


    if (s%lxtra(lx_in_LHe_peak)) then
        target_mesh_delta_coeff = 1.0d0

    else if (log10(abs(s%star_mdot)) > lg_mdot_limit)
     ↪  then
        target_mesh_delta_coeff = 2.0d0

    else
        target_mesh_delta_coeff = 1.0d0
    end if

    if (s%mesh_delta_coeff < target_mesh_delta_coeff)
     ↪  then
        s%mesh_delta_coeff = min( &
            s%mesh_delta_coeff + 0.01d0, &
            target_mesh_delta_coeff)
            write(*,*) 'strong mass loss: increasing
             ↪  mesh_delta_coeff = ', &
                s%mesh_delta_coeff

    else if (s%mesh_delta_coeff >
     ↪  target_mesh_delta_coeff) then
        s%mesh_delta_coeff = max( &
            s%mesh_delta_coeff - 0.01d0, &
            target_mesh_delta_coeff)
            write(*,*) 'TP: decreasing mesh_delta_coeff =
             ↪  ', &
                s%mesh_delta_coeff
    end if

end subroutine resolve_mesh_during_mass_loss
```

d. Effect of wind on q flipping

e. Fixes

Since some of the binaries seem to get stuck in an evolution state with $\log \Delta t < -6$ yr, I think it is important to set some lower limit for the timestep. If it breaches below that, the evolution should stop so the cores on the cluster are freed up.

To see what an appropriate minimum $\log \Delta t$ might be, I plot the minimum value of this quantity for the epsilon grid from a couple of weeks ago.



The most important thing for use now is that the minimum mass never drops below $\log(\Delta t/\mathrm{yr}) = -4$ for the models that terminate and never below -5.5 for the models that undergo the Darwin stability. I think it is safe to set the limit at -6.

The clustering of the models is quite fun too though. It is quite easy to see that the models with a lot of circumbinary mass loss are more likely to experience the Darwin instability, and they will continue RLOF for much longer (in the sense that RLOF only stops after the envelope is stripped down to $10^{-5}\ M_\odot$ in some extreme cases.)

A lower limit of $\log(\Delta t/\mathrm{yr}) = -6$ seems appropriate. I will use this. Since *MESA* expects the lower limit in terms of seconds, this means I will set

```
min_timestep_limit = 30d0 ! seconds, about 10**-6 yr
```

in the common *MESA* inlist.

References

Rees, N. R. & Izzard, R. G. (2024) *Evolving past instabilities on the thermally pulsing-(super)asymptotic giant branch*. MNRAS531, 4033.

A Theo's inlist

```
Common inlist for shared parameters
&star_job
      show_log_description_at_start = .false.


      change_net = .true.
      new_net_name = 'co_burn_extras.net'

      ! age start at ZAMS
      set_initial_age = .false.
      set_initial_model_number = .false.

      num_special_rate_factors = 2
      reaction_for_special_factor(1) = 'r_c12_ag_o16'
      special_rate_factor(1) = 1
      filename_of_special_rate(1) =
       ↪  'c12ag_deboer_sigma_0p0_2000_Tgrid.dat'

      reaction_for_special_factor(2) =
       ↪  'r_he4_he4_he4_to_c12'
      special_rate_factor(2) = 1
      filename_of_special_rate(2) =
       ↪  'r_he4_he4_he4_to_c12_cf88.txt'

/ ! end of star_job namelist

&eos


/ ! end of eos namelist

&kap
      Zbase = 1.2d-02

      kap_file_prefix = 'gs98'
      use_Type2_opacities = .true.

/ ! end of kap namelist

&controls

initial_mass = 5.00d+00

! winds
      cool_wind_full_on_T = 9.99d9
      hot_wind_full_on_T = 1d10
      cool_wind_RGB_scheme = 'Reimers'
      cool_wind_AGB_scheme = 'Blocker'
      RGB_to_AGB_wind_switch = 1d-4
      Reimers_scaling_factor = 0.1d0
      Blocker_scaling_factor = 0.2d0

! when to stop
      max_age = 1.4d+10

! mesh
      max_allowed_nz = 20000 ! preventing 'fail to adjust
       ↪  mesh'

! diffusion
      diffusion_v_max = 1d-5
      do_element_diffusion = .false. ! Activated during
       ↪  MS and settling only
      diffusion_use_full_net = .true.

      ! Solver controls.
      diffusion_use_cgs_solver = .true.
      diffusion_use_isolve = .true.
      diffusion_rtol_for_isolve = 1d-4
      diffusion_atol_for_isolve = 1d-5
      diffusion_min_X_hard_limit = -1d-4
      diffusion_maxsteps_for_isolve = 1000
      diffusion_isolve_solver = 'ros2_solver'

      ! Timestep controls to prevent steps that are
       ↪  difficult for diffusion.
      diffusion_steps_limit = 20
      diffusion_steps_hard_limit = 150
      diffusion_iters_limit = 50
      diffusion_iters_hard_limit = 100

! convection
      mlt_option = 'Henyey'
      mixing_length_alpha = 1.8
      use_Ledoux_criterion = .true.
      thermohaline_coeff = 1d2

! timesteps
      max_years_for_timestep = 1d7
      time_delta_coeff = 1.0d0
      use_gold2_tolerances = .true.

      varcontrol_target = 1d-3
      dX_nuc_drop_limit = 1d-2
      delta_HR_limit = 0.1

! Atm
      atm_option = 'T_tau'
      atm_T_tau_relation = 'Eddington'
      atm_T_tau_opacity = 'iterated'
      atm_T_tau_max_iters = 100

! Overshoot
      overshoot_scheme(1) = 'exponential'
      overshoot_zone_type(1) = 'any'
      overshoot_zone_loc(1) = 'any'
      overshoot_bdy_loc(1) = 'any'
      overshoot_f(1) = 0.014
      overshoot_f0(1) = 0.004

/ ! end of controls namelist

AGB inlist

&star_job

      load_saved_model = .true.
      load_model_filename = 'end_he_core_burn.mod'
      save_model_when_terminate = .true.
      save_model_filename = 'end_AGB.mod'
      required_termination_code_string = ''
      set_initial_model_number = .true.
      initial_model_number = 0
      pgstar_flag = .true.


/ ! end of star_job namelist

&eos

/ ! end of eos namelist

&kap

/ ! end of kap namelist

&controls
      log_directory = 'LOGS_AGB'

! energy conservation
      energy_eqn_option = 'eps_grav'
      use_time_centered_eps_grav = .true.

! when to stop
      envelope_mass_limit = 1e-3

! winds
      max_wind = 1d-2

! solver
      retry_hold = 0
      neg_mass_fraction_hold = 3

! mesh
      mesh_delta_coeff = 1.0

! convection & dragging
      num_cells_for_smooth_gradL_composition_term = 0
      drag_coefficient = 1d0
      min_q_for_drag = 0.98d0
      alpha_semiconvection = 0.1

! timesteps

      varcontrol_target = 1d-4
      delta_lgL_He_limit = 0.005
      lgL_He_burn_min = 2.0
      dX_nuc_drop_limit = 1d-3
      delta_HR_limit = 0.1
      max_timestep_factor = 1.01d0


      delta_lgTeff_limit = 0.005
      delta_lgTeff_hard_limit = 0.01
      delta_lgL_limit = 0.02
      delta_lgL_hard_limit = 0.05


! output
      num_trace_history_values = 2
      trace_history_value_name(1) = 'rel_E_err'
      trace_history_value_name(2) = 'log_rel_run_E_err'

      !report_solver_progress = .true. ! set true to see
       ↪  info about solver iterations
      !report_ierr = .true. ! if true, produce terminal
       ↪  output when have some internal error
      !stop_for_bad_nums = .true.

/ ! end of controls namelist
```